

NetShield AI

Improvement Roadmap

A complete gap analysis of what's missing and what can be improved, organized as a step-by-step checklist to work through starting tomorrow.

How to use this document

Section 1 is a suggested phase-by-phase order — start at Phase 1 and work down; each phase assumes the previous one is roughly done. Section 2 is the complete, uncut list behind those phases, organized by category, with a one-line reason ('Why') for anything non-obvious. Check items off as you go — this file is meant to be marked up, printed, or copied into a tracker.

Section 1 — Suggested Step-by-Step Order

Six phases, roughly in the order I'd tackle them. Phases 1-2 are cheap and high-signal; later phases are bigger undertakings or require infrastructure you may not need yet.

PHASE 1 — Quick, High-Signal Wins

Start here. Small changes, strong payoff for a viva/demo.

- [] Retrain quality gate — don't hot-swap the live model if the new one's metrics are worse
- [] Login rate-limiting / account lockout after repeated failed attempts
- [] A first pytest suite for the backend (10-15 tests covering the core endpoints)
- [] Set GEMINI_API_KEY so both chatbots run at full capability, not degraded mode

PHASE 2 — Portability & Reliability

Solves real pain you've already hit (the Keras version break).

- [] Docker packaging for the backend (and ideally the whole stack)
- [] Database migrations tool (Alembic) instead of create_all()-only schema
- [] Structured logging with request correlation IDs
- [] Continuous live-capture loop wrapper around live_capture_feed.py (rotating tcpdump + auto-feed)

PHASE 3 — Production Hardening

Needed before this is ever exposed beyond your own machine.

- [] HTTPS/TLS termination, then flip the session cookie back to secure=True
- [] Move secrets out of a plaintext .env into a real secrets manager
- [] CSRF protection beyond SameSite=Lax
- [] Rate limiting on /api/chat (open endpoint hitting a paid LLM API)
- [] Security response headers (CSP, X-Frame-Options, HSTS)
- [] Dependency vulnerability scanning (pip-audit / npm audit / Dependabot)
- [] Password policy, reset flow, and (optionally) MFA

PHASE 4 — Scale & Observability

Matters once this handles real traffic volume, not before.

- [] Fix multi-worker model-reload mismatch (retrain on one worker must reload all workers)
- [] Event-driven ingestion (Kafka/RabbitMQ) instead of one-shot HTTP POSTs per chunk
- [] Caching layer (Redis) for hot endpoints like /api/stats/summary
- [] Metrics/telemetry (Prometheus-style) for real p95 latency, throughput, error rates
- [] Alerting on the alerting system itself — page someone if the pipeline breaks or errors spike

- [] Documented database backup/restore strategy
- [] CI/CD pipeline (lint + tests + build on every push)
- [] Load testing and failure-injection testing (DB down mid-request, LLM timeout, etc.)

PHASE 5 — Model & Data Research

Bigger, open-ended investigations — good thesis/report material.

- [] Cross-dataset validation against UNSW-NB15
- [] Parallel (non-sequential) architecture experiment vs. the current gated design
- [] Adversarial robustness testing — can traffic be shaped to evade the anomaly threshold?
- [] Concept-drift monitoring for live traffic vs. training distribution
- [] Confidence calibration check (is 99.99% confidence actually ~99.99% correct?)
- [] Data Exfiltration augmentation, or at minimum a loud UI reliability warning
- [] Cross-window / longer-horizon correlation for slow, patient attacks

PHASE 6 — Polish, Docs & Product

Round out the edges once the core is solid.

- [] Frontend accessibility and mobile-responsiveness audit
- [] Export function (CSV/PDF) for an alert or report from the dashboard
- [] Unmistakable Mock/Demo vs. Live mode indicator in the UI
- [] Operational runbook — what an analyst actually does when a High alert appears
- [] A written threat model for the product itself
- [] Data retention policy for captured traffic
- [] Rules-of-engagement template for live-capture lab testing
- [] Multi-tenancy story if this were ever deployed for more than one organization

Section 2 — Full Gap List, By Category

The complete, uncut list behind the phases above. Items marked (verified) were confirmed directly in the code this session; everything else is a reasoned inference from the current architecture.

1. Detection Quality (the model itself)

[] Cross-dataset validation (UNSW-NB15)

Why: Model has only ever been validated on CICIDS2017.

[] Sequential-gate recall ceiling (~54% end to end)

Why: BiLSTM only ever sees windows the Autoencoder already flagged; a parallel design would raise recall at higher compute cost.

[] No adversarial robustness testing

Why: Unknown whether an attacker can shape traffic to duck under the anomaly threshold.

[] No concept-drift monitoring

Why: Nothing detects the model quietly degrading as live traffic patterns shift over time.

[] Data Exfiltration category has only 36 raw samples

Why: Statistically unreliable; needs augmentation or a loud reliability warning wherever it appears.

[] No confidence calibration check

Why: A 99.99% classifier confidence hasn't been verified to actually mean 99.99% correct.

[] Risk thresholds are fixed

Why: Low/Medium/High cutoffs are calibrated once from validation data; no way to adjust risk appetite per deployment without a full retrain.

[] No cross-window correlation

Why: Detection is per single 10-row window only; a slow attack spread wider than that could evade every window individually.

[] SHAP remains the slowest stage (~670ms mean)

Why: Worth investigating batched SHAP calls or a smaller background sample if live throughput becomes a bottleneck.

2. Data

[] Feature set capped at CICFlowMeter's 76 columns

Why: No domain-engineered additions (payload entropy, protocol-specific signals, IP reputation).

[] No ingest-time validation

Why: Failure mode for a malformed CSV upload isn't documented or verified.

[] No feature-schema versioning

Why: If the feature set changes, old alerts' stored feature JSON has no migration path.

3. Architecture

[] (verified) Retrain hot-swap has no quality gate

Why: Confirmed in retrain.py: any successful retrain unconditionally replaces the live model, even if its metrics are worse than what was already running.

[] Retrain/model-cache is single-worker only

Why: With multiple uvicorn workers, a retrain on one worker wouldn't reload the model on the others.

[] Ingest is synchronous HTTP, not event-driven

Why: A real continuous sensor feed would suit a message queue far better than one-shot POSTs per chunk.

[] No caching layer

Why: Hot endpoints like /api/stats/summary have no cache under real load.

[] No API versioning

Why: Fine today, matters once anything external depends on this.

4. Backend Engineering

[] (verified) Zero automated tests

Why: Confirmed — no test_*.py files anywhere in the repo.

[] No database migrations tool

Why: Schema is create_all()-only; adding a column later has no clean upgrade path.

[] No structured logging / request correlation IDs

Why: Would make a real production issue meaningfully harder to debug.

[] Unverified: no enforced upload size limit on /api/ingest/csv

Why: A huge file could be a memory problem.

[] Offset-based pagination

Why: Fine at current scale, slows down at very large offsets on a big alerts table.

5. Security of the Product Itself

[] (verified) No login rate-limiting or account lockout

Why: Confirmed in auth.py — nothing throttles repeated failed logins.

[] No password policy, reset flow, or MFA

[] (verified) Session cookie is secure=False

Why: Correct for local dev over plain HTTP; must flip before any real HTTPS deployment.

[] Secrets live in a plaintext .env

Why: Fine solo, not fine for an enterprise-wide deployment.

[] No CSRF protection beyond SameSite=Lax

Why: Not a complete defense on its own for state-changing POSTs.

[] No dependency vulnerability scanning

Why: No Dependabot / pip-audit / npm audit currently run.

[] No rate limiting on /api/chat

Why: An open endpoint hitting a paid LLM API, unthrottled, is a real cost/abuse risk.

[] No security response headers

Why: CSP, X-Frame-Options, HSTS all absent.

[] Unverified: prompt-injection surface on the per-alert chatbot

Why: Alert data feeds into its context; worth explicitly testing.

6. Frontend / UX (unverified this session)

[] Accessibility audit

Why: ARIA labels, keyboard navigation, screen-reader support — not yet checked.

[] Mobile responsiveness

Why: Not yet checked.

[] Graceful degradation if the backend is unreachable

Why: Not yet checked.

[] Unmistakable Mock/Demo vs. Live mode indicator

Why: Risk of mistaking demo data for a real alert.

[] No export function

Why: No CSV/PDF export for an alert or report from the dashboard itself.

7. Testing & QA

[] No unit, integration, or end-to-end tests

Why: Neither backend nor frontend has any automated test coverage.

[] No load testing

Why: Backend behaviour under real traffic volume is unknown.

[] No failure-injection testing

Why: E.g. MySQL dropping mid-request, or a Gemini API timeout.

8. DevOps / Deployment

[] (verified) No Docker anywhere in the repo

Why: This is exactly what would have prevented the Keras 2-vs-3 environment break hit earlier.

[] (verified) No CI/CD pipeline

Why: No .github/workflows/ present.

[] No infra-as-code

Why: For the multi-sensor enterprise chokepoint deployment story.

[] No documented DB backup/restore strategy

[] live_capture_feed.py is one-shot, not continuous

Why: Capture → copy → convert → feed is a manual cycle today.

9. Observability

[] No metrics/telemetry

Why: No way to answer 'what's our real p95 latency in production' without ad hoc benchmarking.

[] No alerting on the alerting system itself

Why: If the pipeline breaks or errors spike, nothing currently pages anyone.

10. Documentation / Process

[] No operational runbook

Why: What an analyst actually does when a High alert appears isn't written down.

[] No model changelog or rollback story

Why: Ties directly to the retrain quality-gate gap in Architecture above.

[] No formal threat model for the product itself

11. Product / Compliance

[] No multi-tenancy story

Why: Relevant if this were ever deployed for more than one organization.

[] No data-retention policy for captured traffic

Why: Matters if personal data ever traverses a monitored network.

[] No rules-of-engagement template for live-capture testing

Why: Beyond the informal 'isolated lab only' rule already being followed.

Generated from a full working review of the NetShield AI codebase — nothing above is speculative filler; items marked (verified) were confirmed directly in the code.